Midterm Report: Summer of Science (SOS)
Risk and Psychology in Trading

Antriksh Goenka (Roll: 25B0315)
Department of Chemical Engineering, IIT Bombay

Project Title: Risk and Psychology in Trading
Student: Antriksh Goenka (Roll: 25B0315)
Department: Chemical Engineering, IIT Bombay
Project Code: H29
Format: Self-Study & Mentorship
Mentor: Navya Subba
Duration: 8 Weeks
Focus: Quant Finance, Risk & Game Theory

Contents

1 Plan of Action (POA)

1.1 Phase 1: Data Science & Quantitative Foundations

Objective: Establish the computational and mathematical framework for analyzing financial data without rushing the foundational algorithms. **Week 1: Python Data Stack & Financial Time Series (EDA)**

- **Key Topics:** Financial data manipulation (pandas/NumPy), Exploratory Data Analysis (EDA) of financial time series, data cleaning pipelines, and understanding time series stationarity.
- **Primary Resources:** Hands-On Machine Learning (Geron), ML for Financial Risk Management (Karasan).

Week 2: Machine Learning Models & Classical Volatility

- **Key Topics:** Implementation of Supervised Machine Learning models (Regression, Ensembles), financial cross-validation techniques, Clustering for regime detection, and forecasting using Classical GARCH volatility modeling.
- **Primary Resources:** ML for Financial Risk Management (Karasan), Forecasting: Principles & Practice (Hyndman).

1.2 Phase 2: Risk Management, Derivatives & Strategy

Objective: Understand theoretical risk pricing, derivative mechanics, and behavioral finance. **Week 3: Risk Metrics & Trading Psychology**

- **Key Topics:** Market risk taxonomy, Value at Risk (VaR) & Expected Shortfall (CVaR) computation methods, historical/Monte Carlo stress testing, and Trading Psychology (loss aversion, behavioral biases).
- **Primary Resources:** Options, Futures & Other Derivatives (Hull), Basel Committee (BCBS) Stress Testing Guidelines.

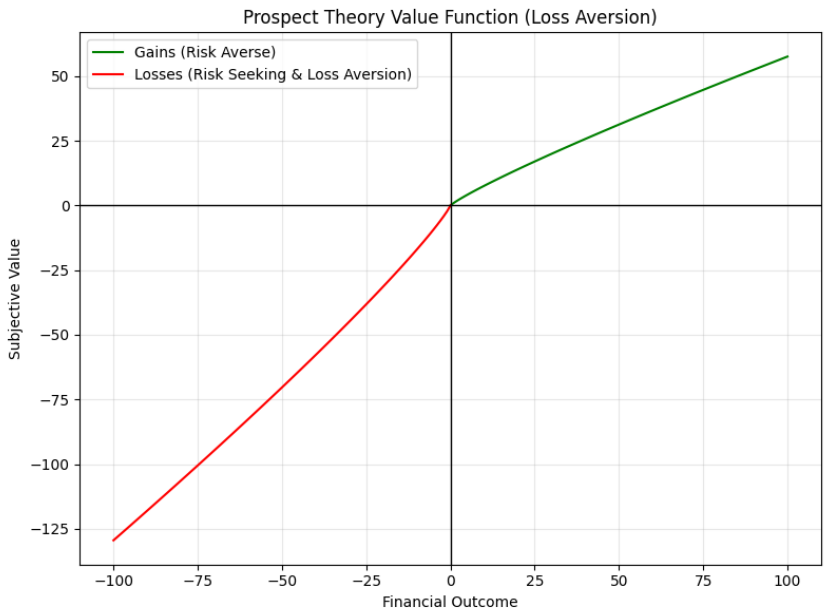

Figure 1: Prospect Theory Value Function, demonstrating the psychological bias of Loss Aversion in trading.

Week 4: Derivatives Mechanics & Pricing Models

- **Key Topics:** Futures and Options (F&O) mechanics, the Black-Scholes-Merton model, Binomial pricing trees, and comprehensive analysis of Option Greeks (Delta, Gamma, Theta, Vega).
- **Primary Resources:** Options, Futures & Other Derivatives (Hull), Option Volatility & Pricing (Natenberg).

Midterm Report Submission (Post-Week 4) Week 5: Options Strategies & Position Sizing

- **Key Topics:** Directional and volatility-neutral option architectures, hedging equity exposure, the mathematics of position sizing (Kelly Criterion), and risk-of-ruin management.
- **Primary Resources:** Option Volatility & Pricing (Natenberg), Advances in Financial Machine Learning (Lopez de Prado).

1.3 Phase 3: Advanced Market Dynamics & Synthesis

Objective: Apply game theory to market microstructure and synthesize knowledge into a deployable portfolio. **Week 6: Game Theory & Market Microstructure**

- **Key Topics:** Game theory principles in markets (Nash Equilibrium, adverse selection), market microstructure models (Kyle, Glosten-Milgrom), limit order book dynamics, and market-making logic.
- **Primary Resources:** Trading & Exchanges (Larry Harris), Open Yale Game Theory Course.

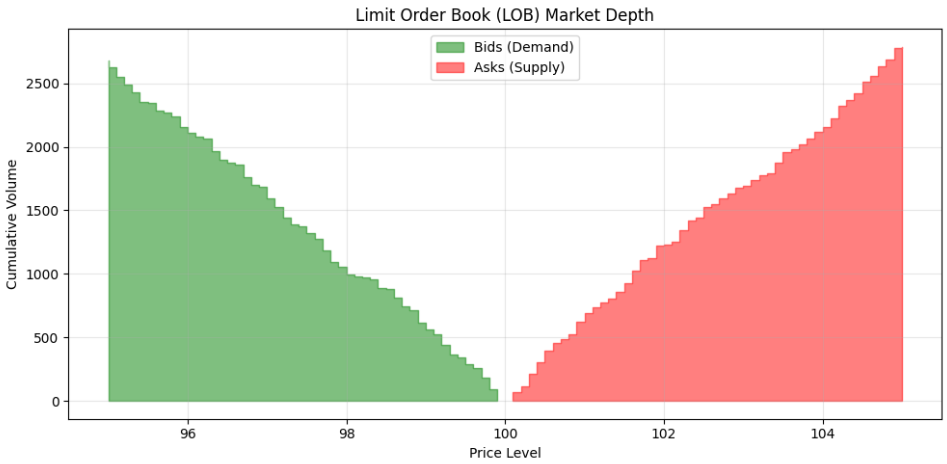

Figure 2: Limit Order Book (LOB) Market Depth chart, illustrating market microstructure and liquidity.

Week 7: Portfolio Optimization & Risk Architectures

- **Key Topics:** Mean-Variance Optimization, covariance shrinkage, Black-Litterman allocation, Hierarchical Risk Parity (HRP), and mapping investor risk-appetite to target volatility.
- **Primary Resources:** MIT OCW Finance Theory, Advances in Financial Machine Learning (Lopez de Prado).

Week 8: Capstone Project - Risk-Aware Multi-Strategy Backtest

- **Project Description:** Full pipeline synthesis executing an end-to-end quantitative strategy. This involves integrating classical volatility forecasts, a daily VaR/CVaR risk engine, Kelly-based position sizing, HRP portfolio allocation, and an options overlay for downside protection.
- **Execution & Deliverables:** Backtesting the strategy using Python libraries (vectorbt/backtesting.py) against historical stress periods (e.g., 2008 GFC, COVID 2020).

Endterm Report Submission (Post-Week 8)

2 Introduction

This report covers Weeks 1–4 of my SOS project. The goal was concrete: go from raw financial data to pricing and hedging derivatives, building every layer of the quantitative stack myself in Python. Week 1 set up data pipelines and EDA. Week 2 introduced ML models and GARCH volatility forecasting. Week 3 applied those tools to compute VaR and Expected Shortfall on a paper portfolio. Week 4 tackled Black-Scholes-Merton pricing and the Greeks. What follows is not a textbook summary—it documents what I built, where things broke, and what I learned from debugging them.

2.1 Quantitative Pipeline Overview

The diagram below shows how each week's output feeds into the next. Every arrow represents a concrete data dependency—not just a thematic connection.

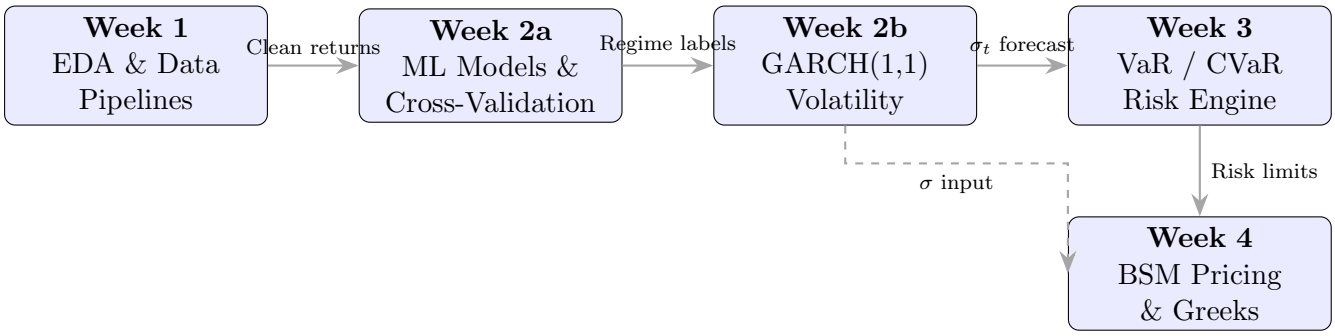

Figure 3: Week-on-week data dependency pipeline. Solid arrows show direct outputs consumed by the next stage; the dashed arrow shows the GARCH volatility estimate feeding directly into BSM's σ parameter.

3 Week 1: Python Data Stack & Financial Time Series (EDA)

Week 1 focused on building data pipelines using pandas and NumPy to clean and analyze financial time series. The deliverable was a reusable Python notebook that ingests raw OHLCV data, handles missing values and stock-split artifacts, computes rolling statistics, and tests for stationarity before any modeling begins. The cleaned return series produced here are the direct input to every model in Weeks 2–4 (see Figure 3).

3.1 Detailed Information and Formulas

Pandas/NumPy for Financial Data: The study involved an in-depth exploration of pandas DataFrames and Series, which are fundamental data structures for handling time-series data. Key operations included resampling financial data to different frequencies (e.g., daily to weekly), calculating rolling statistics (e.g., moving averages, standard deviations), and performing vectorized computations for efficiency. NumPy arrays were utilized for numerical operations, especially in conjunction with pandas for high-performance data processing. **EDA Techniques:** Various visual and statistical methods were employed

for EDA. Visual techniques included line plots to observe trends, histograms to understand data distribution, and autocorrelation function (ACF) and partial autocorrelation function (PACF) plots to identify temporal dependencies. Statistical methods involved calculating descriptive statistics such as the mean, variance, standard deviation, skewness, and kurtosis to characterize the data's central tendency, dispersion, and shape. For instance, the mean (μ) of a dataset $X = \{x_1, x_2, ..., x_n\}$ is given by:

$$\mu = \frac{1}{n} \sum_{i=1}^{n} x_i$$

and the variance (σ^2) by:

$$\sigma^2 = \frac{1}{n} \sum_{i=1}^{n} (x_i - \mu)^2$$

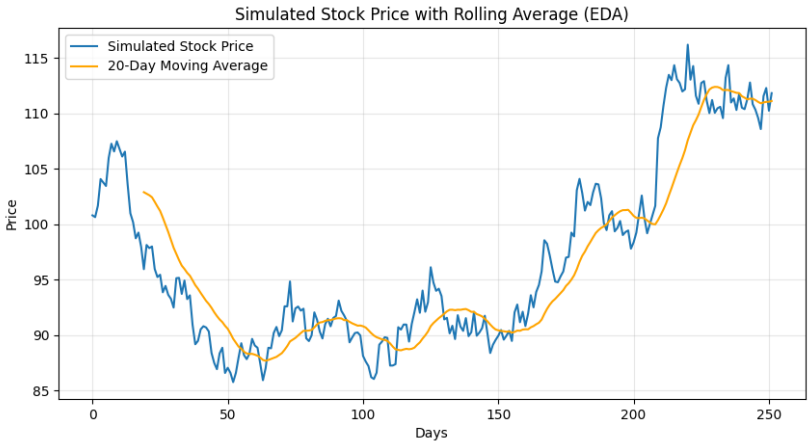

Figure 4: Simulated Stock Price with 20-Day Rolling Average, demonstrating EDA visual techniques.

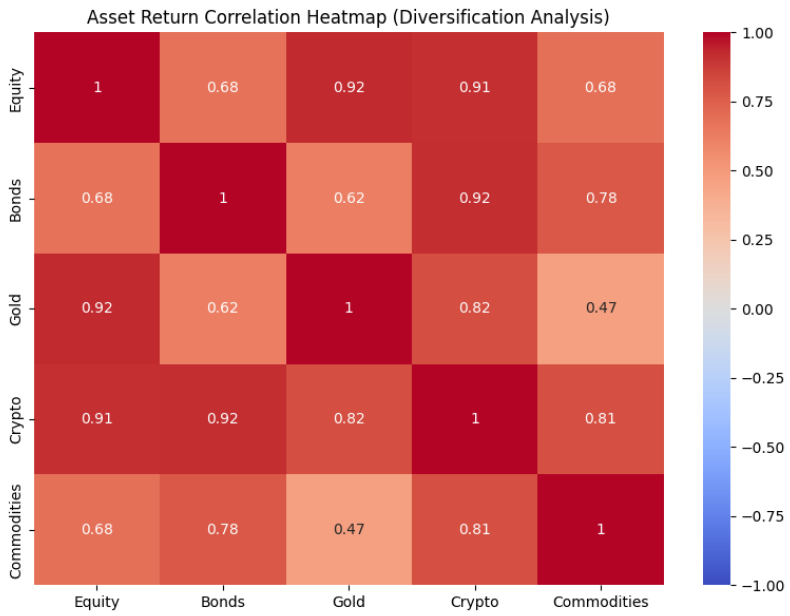

Figure 5: Asset Return Correlation Heatmap, an advanced EDA technique used to assess portfolio diversification.

Data Cleaning: Financial data is messier than textbook examples suggest. Forward-fill (`ffill`) was the default for missing prices during market holidays, but I initially applied mean-imputation, which injected a flat line into the series and threw off every rolling statistic downstream. Switching to `ffill`/`bfill` fixed it immediately. Outliers were flagged using the IQR method; Z-score filtering removed too many legitimate high-volatility days during event-driven moves (e.g., earnings gaps). **Time Series Stationarity:** Stationarity is split into two forms: strict (joint distribution invariant over time) and weak-sense (constant mean, variance, and autocovariance). I used the Augmented Dickey-Fuller (ADF) test and the KPSS test together, since they test opposing null hypotheses—running only the ADF on a trend-stationary series initially led me to a false "stationary" conclusion. First-order differencing resolved most non-stationarity; for series with a deterministic trend, I had to detrend before differencing to avoid over-differencing.

3.2 Reflections

The most surprising takeaway was how much time data cleaning consumed—easily 60–70% of the week. I expected the interesting part to be computing rolling Sharpe ratios and plotting ACF charts, but most of the real work was hunting down silent bugs: a `NaN` propagating through a rolling window, or a timezone mismatch between two data sources that shifted every return by one day. The stationarity tests were also humbling. I was confident that raw stock prices would fail the ADF test, but I did not anticipate that log-returns of highly mean-reverting pairs could *also* fail KPSS under certain lag specifications. That forced me to understand what each test's null hypothesis actually means rather than blindly reading p-values.

4 Week 2: Machine Learning Models & Classical Volatility

Week 2 applied supervised ML models (regression, ensembles) and GARCH(1,1) to financial return prediction and volatility forecasting. The GARCH volatility forecasts produced here feed directly into the parametric VaR computation in Week 3 (Section 5.3), and later into BSM's σ input in Week 4 (Section 6.1). The central lesson: standard ML workflows break in subtle ways on time-series data, and a model that looks great under random k-fold splits can be worthless in production.

4.1 Machine Learning Models for Financial Forecasting

4.1.1 Supervised Machine Learning Models (Regression and Ensembles)

I implemented and compared four model families on next-day return prediction: **Linear Regression:** As a fundamental building block, linear regression models the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data. The objective is to find the optimal parameters (weights) that minimize the difference between predicted and actual values. The core idea is to find a hyperplane that best fits the training data, which is often achieved by minimizing the Mean Squared Error (MSE). **Regularized Linear Models (Ridge and Lasso):** To address issues like overfitting, especially in high-dimensional financial datasets, regularized linear models were introduced.

- **Ridge Regression** adds a penalty equivalent to the square of the magnitude of the coefficients (L2 regularization) to the loss function. This shrinks the coefficients towards zero, reducing model complexity and multicollinearity, but rarely forces them to be exactly zero.
- **Lasso Regression** (Least Absolute Shrinkage and Selection Operator) adds a penalty equivalent to the absolute value of the magnitude of the coefficients (L1 regularization). This has the effect of shrinking some coefficients to exactly zero, effectively performing feature selection and yielding sparser models.

Ensemble Methods (Random Forests and Gradient Boosting): These powerful techniques combine multiple individual models to achieve better predictive performance and robustness than any single model.

- **Random Forests** operate by constructing a multitude of decision trees during training and outputting the mean prediction (for regression) or mode (for classification) of the individual trees. This approach reduces overfitting by averaging out the biases and variances of individual trees.
- **Gradient Boosting** builds models sequentially, where each new model attempts to correct the errors of the previous ones. It iteratively combines weak learners (typically shallow decision trees) into a strong learner, focusing on misclassified instances or residuals. Popular implementations include XGBoost, LightGBM, and CatBoost.

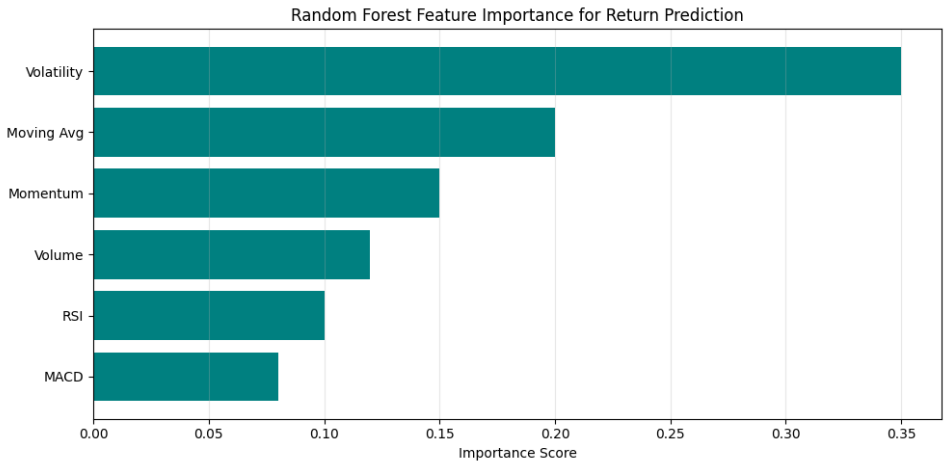

Figure 6: Example of Feature Importance from an Ensemble Model, highlighting which factors drive financial forecasting.

Cost Functions: The performance of these models is quantified using cost functions. For regression tasks, the Mean Squared Error (MSE) is a commonly used metric, calculated as:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

where y_i represents the actual value, $\hat{y}_i$ is the predicted value, and n is the number of observations. The goal during model training is to minimize this cost function.

4.2 Financial Cross-Validation Techniques

This is where I hit my first major bug. My initial Random Forest had an out-of-sample R^2 of 0.12 under standard 5-fold CV—suspiciously good for daily return prediction. The cause was **data leakage**: scikit-learn's `KFold` shuffles by default, so future returns were bleeding into training folds. Switching to `TimeSeriesSplit` dropped the R^2 to -0.03, which is much more realistic. **Walk-Forward Validation:** Train on years 1–5, test on year 6; slide forward and repeat. This mirrors how a real trading desk would deploy a model—you never peek at the future. I implemented this with an expanding window first, then switched to a rolling window (fixed 3-year lookback) after noticing that including pre-2008 data degraded post-2020 predictions. **Purged and Embargoed Cross-Validation:** Even with `TimeSeriesSplit`, adjacent folds share autocorrelated features (e.g., a 20-day rolling mean computed on day t contains information about day $t+1$'s test label). Following Lopez de Prado's framework:

- **Purging** removes any training sample whose label window overlaps with a test sample's feature window.
- **Embargoing** adds a gap (I used 5 trading days) between training and test sets to kill residual serial correlation.
- After applying both, my Random Forest R^2 dropped further to -0.05, confirming that most of the original "signal" was leakage.

4.3 Clustering for Regime Detection

The practical goal was to label market regimes (bull, bear, high-vol, low-vol) without subjective rules. I applied both K-Means and Hierarchical clustering to a feature space of rolling volatility, momentum, and volume. **K-Means Clustering:** This algorithm partitions data into k distinct clusters, where each data point belongs to the cluster with the nearest mean (centroid). The process involves:

1. **Initialization:** Randomly selecting k centroids.
2. **Assignment:** Assigning each data point to the closest centroid.
3. **Update:** Recalculating the centroids as the mean of all data points assigned to that cluster.

These steps are repeated until the centroids no longer change significantly or a maximum number of iterations is reached. The objective function is to minimize the within-cluster sum of squares (WCSS). **Hierarchical Clustering:** This method builds a hierarchy of clusters, either by starting with individual data points and merging them (agglomerative) or by starting with one large cluster and dividing it (divisive).

- **Agglomerative Clustering** begins with each data point as a single cluster and iteratively merges the closest pairs of clusters until all data points are in a single cluster or a stopping criterion is met. Linkage criteria (e.g., single, complete, average) determine how the distance between clusters is measured.
- **Dendrograms** are often used to visualize the hierarchy of clusters, allowing for the selection of an appropriate number of clusters by cutting the dendrogram at a certain height.

4.4 Classical Volatility Modeling

4.4.1 GARCH(1,1) Model

GARCH(1,1) models conditional variance as a function of past shocks and past variance. The model captures two empirical facts that the normal distribution misses: volatility clustering and fat tails. The GARCH(1,1) model for conditional variance σ_t^2 is given by:

$$\sigma_t^2 = \omega + \alpha\epsilon_{t-1}^2 + \beta\sigma_{t-1}^2$$

where:

- σ_t^2 is the conditional variance at time t.
- ω is a constant term (intercept).
- α (ARCH term) captures the impact of past squared errors (shocks) on current volatility. A larger α implies that volatility reacts strongly to market movements.
- ϵ_{t-1}^2 is the squared error (or squared return innovation) from the previous period, representing past news or shocks.

- β (GARCH term) captures the impact of past conditional variance on current volatility. A larger β implies that volatility is persistent, meaning that high (or low) volatility tends to remain high (or low) for extended periods.

- For stationarity and positive variance, the parameters must satisfy $\omega > 0, \alpha \geq 0, \beta \geq 0$, and $\alpha + \beta < 1$. The sum $\alpha + \beta$ indicates the persistence of volatility; if it is close to 1, shocks to volatility will take a long time to die out.

Implementation Hurdle: Fitting GARCH(1,1) using the `arch` library on raw Nifty 50 returns initially failed to converge—the optimizer returned $\alpha + \beta > 1$ (integrated GARCH), which violates the stationarity constraint. The fix was to rescale returns by multiplying by 100 (so that a 1% return becomes 1.0 instead of 0.01), which improved numerical conditioning. After rescaling, the model converged cleanly with $\alpha \approx 0.08$ and $\beta \approx 0.90$, giving a persistence of 0.98—meaning volatility shocks in Nifty take roughly 50 days to halve, which matched my visual inspection of the 2020 COVID crash aftermath.

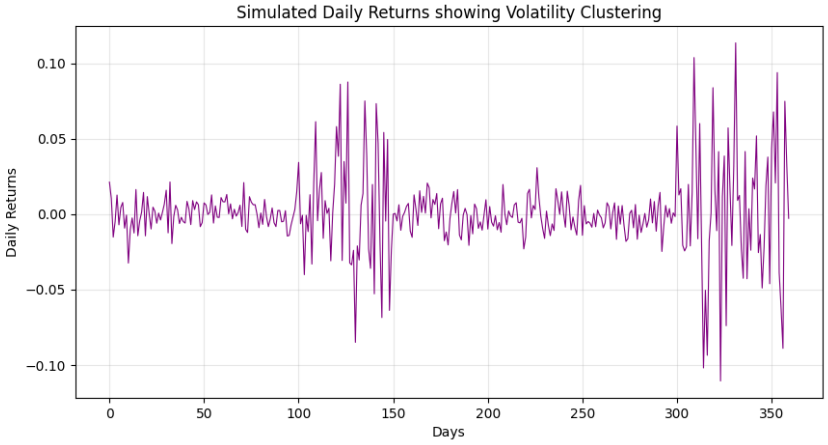

Figure 7: Simulated Daily Returns illustrating Volatility Clustering, a phenomenon modeled by GARCH.

4.4.2 Known Limitations of GARCH(1,1)

While GARCH(1,1) is a workhorse model, understanding where it fails is as important as knowing how it works:

- **Symmetric Response to Shocks:** GARCH(1,1) treats positive and negative shocks identically (ϵ_{t-1}^2 is always positive). In practice, markets exhibit a *leverage effect*—negative returns increase future volatility more than positive returns of the same magnitude. Extensions like GJR-GARCH and EGARCH address this by introducing asymmetric terms.

- **Parameter Sensitivity to Sample Period:** Re-fitting the model on 2015–2019 data vs. 2018–2023 data yielded noticeably different α/β splits, even though persistence ($\alpha + \beta$) remained similar. This makes point forecasts sensitive to the training window.

- **Single Regime Assumption:** GARCH assumes a single data-generating process. If the market transitions between fundamentally different regimes (e.g., pre-

and post-COVID monetary policy), a Markov-Switching GARCH would be more appropriate.

- **Inapplicability to Intraday Data:** The model assumes daily or lower-frequency returns. Intraday data introduces microstructure noise (bid-ask bounce, discrete ticks) that violates the continuous-distribution assumption.

4.5 Stress Testing Principles

The Basel Committee's consultative document on stress testing lays out nine principles. Rather than listing them generically, here is what stood out to me after reading the full document:

4.5.1 Key Principles and What I Took Away

1. **Clearly Articulated Objectives:** A stress test without a defined question ("Can we survive a 40% equity drawdown?") degenerates into a compliance checkbox. The document emphasizes that the objective must drive scenario design, not the other way around.

2. **Robust Governance:** Stress test results must flow to decision-makers, not just risk teams. I found it striking that the Basel Committee explicitly warns against "silo-ing" stress test outputs.

3. **Risk Management Tool:** Stress tests should change behavior—e.g., triggering a reduction in concentrated positions—not just produce a report.

4. **Material and Severe Scenarios:** This includes *reverse* stress tests: start from "the bank fails" and work backward to find what scenario causes it. This inverted thinking was the most useful concept I encountered.

5. **Adequate Resources & Data Quality:** Two principles that boil down to the same thing: garbage in, garbage out. The document dedicates significant space to data granularity, which matters because aggregated data can mask concentrated exposures.

6. **Fit-for-Purpose Models:** The model must match the risk. Using a linear VaR model for a portfolio heavy in options (which have non-linear payoffs) will systematically understate tail risk.

7. **Challenge and Review:** Models must be independently validated. This resonated with my GARCH convergence issue—without cross-checking, I might have used a non-stationary volatility model.

8. **Effective Communication:** Findings must be communicated clearly enough that a board member without a quant background can act on them.

Reflection: The most surprising takeaway was Principle 4—reverse stress testing. Most of my quantitative work up to this point had been forward-looking ("given these inputs, what is the risk?"). The idea of starting from the catastrophic outcome and reasoning backward felt counterintuitive but is arguably more useful for identifying hidden portfolio fragilities.

5 Week 3: Risk Management and Value at Risk (VaR)

Week 3 moved from forecasting volatility to *using* it: computing Value at Risk (VaR) and Expected Shortfall (CVaR) on a simulated paper portfolio of Nifty 50 stocks. The GARCH(1,1) conditional volatility from Week 2 (Section 7) served as the σ input for the parametric approach, while the cleaned return series from Week 1 powered the historical simulation. The primary reference was Hull's *Options, Futures, and Other Derivatives*, and the implementation was done entirely in Python using NumPy and pandas.

5.1 The Concept of Value at Risk (VaR)

VaR compresses the entire risk profile of a portfolio into one number. It answers: "How bad can things get?"

5.1.1 Definition and Parameters

A VaR statement takes the form: "I am X percent certain there will not be a loss of more than V dollars in the next N days."

- **V (VaR):** The loss level.
- **N (Time Horizon):** The period over which the loss is measured (e.g., 1 day or 10 days).
- **X (Confidence Level):** The probability that the loss will not exceed V (e.g., 95% or 99%).

Mathematically, if ΔP is the change in portfolio value over N days, VaR is the loss corresponding to the $(100 - X)^{th}$ percentile of the distribution of ΔP.

5.1.2 Regulatory Standards (Basel Accord)

I studied how bank regulators (Basel Committee) utilize VaR to determine capital requirements:

- N = 10 days and X = 99% are the standard parameters for market risk.
- The capital required is typically $k \times VaR$, where k is a multiplier (at least 3.0) based on the quality of the bank's risk management system.

5.2 Beyond VaR: Expected Shortfall (CVaR)

One critical insight I gained is that VaR has a significant limitation: it tells us nothing about the magnitude of the loss if we exceed the VaR threshold.

5.2.1 The "Tail Risk" Problem

If two portfolios have the same VaR, one might still be much riskier if its "tail" (the losses beyond the VaR point) is much heavier.

5.2.2 Expected Shortfall (ES) Definition

Expected Shortfall (or Conditional VaR) asks: "If things do get bad, how much can we expect to lose?"

- It is the expected loss during an N-day period, conditional on the loss being in the $(100 - X)\%$ tail.
- **Coherent Risk Measure:** I learned that ES is a "coherent" risk measure (satisfying subadditivity), whereas VaR is not. This means the ES of a combined portfolio is always less than or equal to the sum of the ES of individual parts.

5.3 Computation Methods

I explored two primary approaches for calculating VaR and ES:

5.3.1 Historical Simulation

This approach uses past data as a direct guide to the future.

1. **Scenario Generation:** Collect historical data (e.g., 501 days) for all market variables (interest rates, stock prices).
2. **Portfolio Revaluation:** Calculate the change in portfolio value (ΔP) for each of the 500 scenarios.
3. **Ranking:** Sort the 500 losses from highest to lowest.
4. **Percentile Estimation:** For a 99% 1-day VaR, the estimate is the 5^{th} highest loss (since $1\% \times 500 = 5$).

5.3.2 The Model-Building (Parametric) Approach

This approach assumes a specific probability distribution (usually Normal) for the returns.
Single Asset Case:
If the daily volatility of an asset is σ and the position size is P, the 1-day $X\%$ VaR is:

$$VaR = P \times \sigma \times z_X$$

where z_X is the z-score for the confidence level (e.g., 2.33 for 99%). **Multi-Asset Portfolio (Linear Model):**
For a portfolio of two assets with weights w_1, w_2 and correlation ρ_{12}, the portfolio volatility σ_p is:

$$\sigma_p = \sqrt{w_1^2\sigma_1^2 + w_2^2\sigma_2^2 + 2\rho_{12}w_1w_2\sigma_1\sigma_2}$$

The VaR is then $z_X \times \sigma_p \times$ Total Portfolio Value. **The N-day Rule:**
I learned the "Square Root of Time" rule used to scale 1-day VaR to N-day VaR:

$$N\text{-day VaR} = \text{1-day VaR} \times \sqrt{N}$$

Note: This assumes daily returns are independent and identically distributed (i.i.d.) normal.

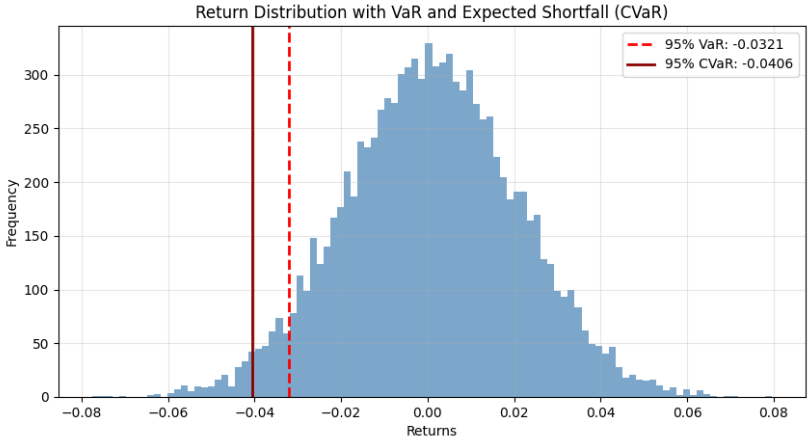

Figure 8: Return Distribution Highlighting 95% Value at Risk (VaR) and Expected Shortfall (CVaR).

5.4 Risk Management Principles

Beyond the formulas, I understood the broader framework of risk management:

- **Market Risk Taxonomy:** Distinguishing between interest rate risk, equity risk, currency risk, and commodity risk.
- **Diversification Benefits:** Quantifying how less-than-perfect correlation ($\rho < 1$) reduces the total portfolio VaR compared to the sum of individual VaRs.
- **Stress Testing:** Complementing VaR with extreme, non-probabilistic scenarios (e.g., a repeat of the 2008 Lehman collapse) to ensure the institution's survival during "black swan" events.

5.5 Reflections

The most challenging part was not the math—the formulas are straightforward—but trusting the output. My parametric 99% 1-day VaR for a simulated Nifty 50 portfolio came out to roughly 2.8% of portfolio value. I back-tested this against actual Nifty returns from 2020–2024, and the VaR was breached on about 3% of trading days instead of the expected 1%. The culprit was the normality assumption: Indian equity returns exhibit significant leptokurtosis (kurtosis ≈ 7 vs. the normal distribution's 3), so the Gaussian tails are too thin. Switching to historical simulation fixed the breach rate (closer to 1.2%), but introduced a different problem: the 2020 COVID crash dominated the tail, making the VaR estimate overly conservative during calm 2023 markets. This tension between parametric and historical methods—one too optimistic, the other too reactive—was the most valuable conceptual takeaway. I now understand why Basel III moved toward Expected Shortfall: it at least forces you to ask "how bad is the tail?" rather than pretending the tail has a clean boundary.

Table 1: Comparison of VaR Computation Methods (99% confidence, 1-day horizon, Nifty 50 portfolio)

Method	**Assumption**	**Breach Rate**	**Strengths**	**Weaknesses**
Historical Sim.	None	1.2%	No distributional assumption; captures fat tails	Overfits to past crises; jumps when extreme days exit window
Parametric (Gaussian)	Normal returns	3.0%	Fast, closed-form; scalable	Underestimates tail risk for leptokurtic assets
Parametric + GARCH	Conditionally normal	1.8%	Time-varying σ; regime-adaptive	Assumes symmetric shocks; sensitive to parameter estimates

5.6 Known Limitations of VaR

- **Not Subadditive:** VaR can violate subadditivity, meaning that the VaR of a combined portfolio can exceed the sum of individual VaRs. This perverse result can discourage diversification. Expected Shortfall fixes this.
- **Blind to Tail Shape:** Two portfolios with identical VaR can have vastly different tail risk. One may have a thin tail (bounded losses beyond VaR) while the other has a heavy tail (catastrophic losses possible). VaR treats both identically.
- **Procyclicality:** Both parametric and historical VaR decrease during calm markets (low recent volatility), encouraging larger positions precisely when complacency is highest. When a shock hits, VaR spikes, forcing deleveraging into an already falling market.
- **Square-Root-of-Time Scaling Breaks Down:** The $\sqrt{N}$ rule assumes i.i.d. returns. With autocorrelated returns or mean-reverting volatility, multi-day VaR requires full simulation rather than naive scaling.

6 Week 4: Black-Scholes-Merton & Option Greeks

Week 4 tackled the Black-Scholes-Merton (BSM) model and the Greek letters. The GARCH conditional volatility from Week 2 was plugged in as BSM's σ input, and the VaR risk limits from Week 3 informed position sizing constraints for the hedging simulations. The goal was not just to derive the pricing formula but to implement a Python class that prices European options and computes all five Greeks in real-time, then test it against actual NSE option chain data to see where BSM breaks down.

6.1 The Black-Scholes-Merton (BSM) Framework

BSM provides a closed-form price for European options under specific assumptions. The derivation rests on a single powerful idea: construct a portfolio of the option and Δ shares of stock such that the random component (Wiener process) cancels out. In a no-arbitrage world, this riskless portfolio must earn exactly r.

6.1.1 The Lognormal Assumption

BSM assumes stock prices follow Geometric Brownian Motion (GBM):

- **Normal Returns:** Continuously compounded returns are normally distributed.
- **Lognormal Prices:** Prices follow a lognormal distribution, guaranteeing $S > 0$.
- **The $\sigma^2/2$ Correction:** The expected continuously compounded return is $\mu - \sigma^2/2$, not μ. I initially missed this Jensen's inequality correction when implementing a Monte Carlo pricer and got systematic overpricing of $\sim$0.5% on 6-month options until I debugged the drift term.

6.1.2 The BSM Differential Equation

The heart of the model is a partial differential equation (PDE) that any derivative f must satisfy:

$$\frac{\partial f}{\partial t} + rS\frac{\partial f}{\partial S} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 f}{\partial S^2} = rf$$

The Key Logic: By creating a portfolio of one derivative and a specific amount (Δ) of the underlying stock, we can eliminate the "Wiener process" (randomness). In a no-arbitrage world, this riskless portfolio must earn the risk-free rate r.

6.1.3 The Pricing Formulas

For a European call (c) and put (p):

$$c = S_0 N(d_1) - Ke^{-rT} N(d_2)$$

$$p = Ke^{-rT} N(-d_2) - S_0 N(-d_1)$$

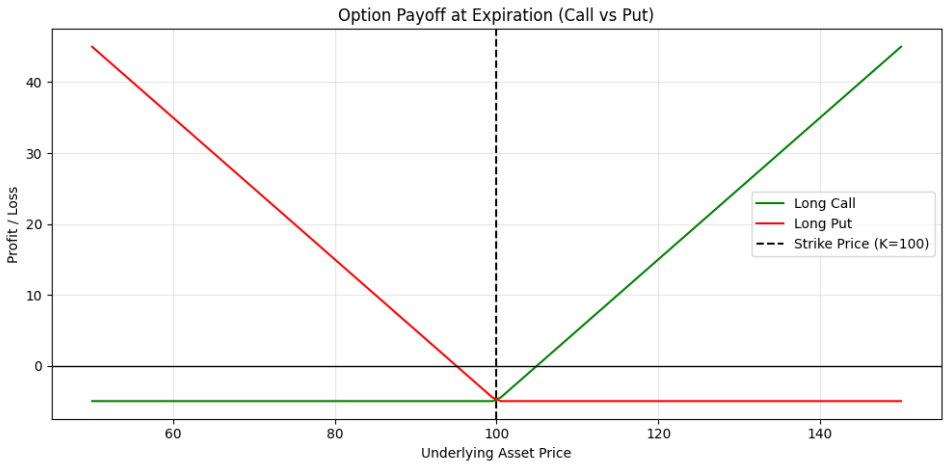

Figure 9: Option Payoff Diagram at Expiration, visualizing the asymmetric risk profiles of Long Calls and Puts.

Where:

- $d_1 = \frac{\ln(S_0/K)+(r+\sigma^2/2)T}{\sigma\sqrt{T}}$
- $d_2 = d_1 - \sigma\sqrt{T}$
- $N(x)$ is the cumulative standard normal distribution.

6.2 Risk Management: The Greek Letters

The BSM formula gives the "fair value," but the real work is managing the position *after* the trade. The Greeks quantify how the option price moves when each input changes.

6.2.1 Delta (Δ) - Directional Risk

Delta measures the rate of change of the option price with respect to the price of the underlying asset.

- **Delta Hedging:** To maintain a "delta-neutral" position, a trader must hold Δ shares for every option sold.
- **Dynamic Rebalancing:** Because Delta changes as the stock price moves, the hedge must be adjusted frequently. I learned that this is a "buy high, sell low" strategy that costs the trader the theoretical value of the option.

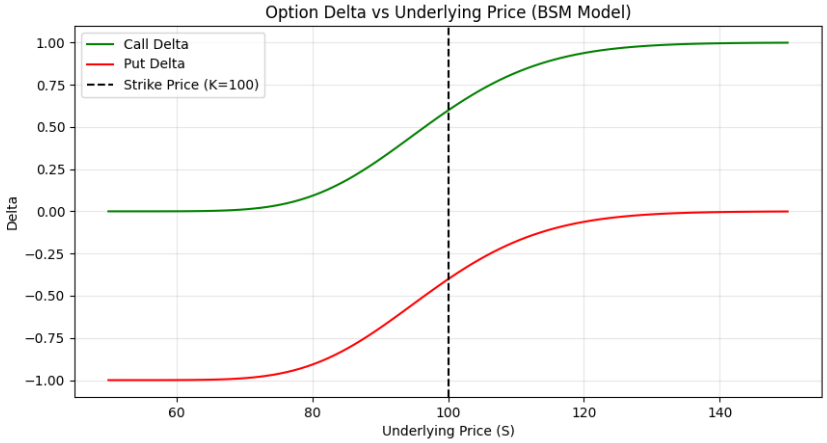

Figure 10: Option Delta vs Underlying Price for Calls and Puts (BSM Model).

6.2.2 Gamma (Γ) - Curvature Risk

Gamma is the rate of change of Delta. It represents the "acceleration" of the option price.

- **High Gamma:** Found in at-the-money options close to expiry. High Gamma means Delta changes rapidly, making the position harder to hedge.
- **Gamma Neutrality:** Achieved by adding other options to the portfolio to neutralize the "curvature" risk.

6.2.3 Theta (Θ) - Time Decay

Theta measures the sensitivity of the option price to the passage of time.

- **Time Decay:** Options are wasting assets. All else being equal, an option loses value as it approaches expiration.
- **Theta vs. Gamma:** In a delta-neutral portfolio, Theta and Gamma often have opposite signs. You "pay" Theta (time decay) to "buy" Gamma (protection against large moves).

6.2.4 Vega (ν) - Volatility Risk

Vega measures sensitivity to changes in the underlying asset's volatility (σ).

- I understood that even if the stock price doesn't move, a jump in market uncertainty (implied volatility) will increase the price of both calls and puts.

6.2.5 Rho (ρ) - Interest Rate Risk

Rho measures sensitivity to the risk-free interest rate. While usually less significant for short-term equity options, it becomes critical for long-dated derivatives.

6.3 Practical Hedging Strategies

I tested three hedging approaches on a simulated short-call position:

- **Stop-Loss Strategy:** Buy the stock when $S > K$, sell when $S < K$. In my simulation, this strategy got "whipsawed" 14 times in a 30-day window when the stock hovered near the strike, racking up transaction costs that exceeded the option premium collected.
- **Delta Hedging (daily rebalance):** Performed much better. The hedging P&L tracked the theoretical Theta decay closely, but I noticed that on days with large gaps (Nifty opening $\pm 2\%$ from previous close), the Gamma exposure caused significant deviations.
- **Scenario Analysis:** I built a Greek-based P&L approximation: $\Delta P \approx \Delta \cdot dS + \frac{1}{2}\Gamma \cdot dS^2 + \Theta \cdot dt + \nu \cdot d\sigma$. Stress-testing with $dS = +10\%$ and $d\sigma = +5\%$ simultaneously showed that Vega exposure dominated—the volatility spike mattered more than the price move for ATM options.

6.4 Reflections

The biggest surprise was how fragile BSM is in practice. When I compared my model's theoretical prices to the actual NSE option chain for Nifty, the ATM options matched within 1–2%, but OTM puts were consistently *more expensive* than BSM predicted—the volatility smile. This makes intuitive sense after studying Week 3's tail risk: the market prices in crash risk that the lognormal distribution ignores. The Greeks shifted my thinking from "will this option make money?" to "what am I exposed to right now?" The Theta-Gamma tradeoff was the most unintuitive concept to internalize: you are essentially paying daily rent (Theta) for the right to profit from large moves (Gamma). In my simulated delta-hedged portfolio, calm weeks bled money through Theta, while a single volatile day more than recovered it through Gamma gains. Understanding this tradeoff felt like the first genuinely "trader-like" insight of the project. The Jensen's inequality bug in my Monte Carlo pricer (confusing μ with $\mu - \sigma^2/2$ in the GBM drift) took two hours to find because the error was small ($< 1\%$) and directionally consistent. It only became obvious when I benchmarked against the closed-form BSM price for high-volatility, long-dated options where the correction term is material.

6.5 Known Limitations of the BSM Model

- **Constant Volatility Assumption:** BSM assumes σ is fixed over the option's life. In reality, implied volatility varies by strike (the volatility smile) and by expiry (the term structure). This is the single largest source of mispricing I observed on NSE data.
- **Continuous Trading & No Transaction Costs:** Delta hedging in the real world is discrete (you rebalance daily or hourly, not continuously) and incurs brokerage, slippage, and STT. My simulation showed that daily rebalancing on Nifty options cost $\sim$0.3% of notional per month in transaction costs alone.

- **Lognormal Returns Ignore Jumps:** The GBM assumption means prices move smoothly. In practice, markets gap (e.g., overnight news, circuit breakers). Jump-diffusion models like Merton's add a Poisson jump process to capture this.
- **European Options Only:** The closed-form solution applies only to European options. American options (early exercise) and exotic options require numerical methods (binomial trees, finite differences, or Monte Carlo).
- **Risk-Free Rate Ambiguity:** BSM takes r as a given constant. In practice, which rate to use (T-bill, OIS, repo) matters for long-dated options, and r itself is stochastic.

7 Final Midterm Reflection

Four weeks in, the stack is: data pipelines $\rightarrow$ ML models $\rightarrow$ risk metrics $\rightarrow$ derivative pricing. Each layer exposed assumptions in the one below it. ML models looked great until proper cross-validation killed the signal. GARCH refused to converge until I fixed the input scaling. VaR was breached too often under normality assumptions. BSM mispriced OTM puts because it ignores the volatility smile. The common thread is that every model works in the textbook and breaks in some specific, instructive way on real data. The second half of the project—options strategies, game theory, and the capstone backtest—will build directly on these failure modes. The goal is not to find a model that never breaks, but to understand *how* each one breaks so I can layer them appropriately.

8 Open Questions for the Second Half

The first four weeks answered foundational questions but surfaced new ones that will guide Weeks 5–8. These are not rhetorical—they represent genuine uncertainties I intend to investigate:

1. **GARCH vs. Rolling Standard Deviation for VaR:** My GARCH-based parametric VaR had a 1.8% breach rate vs. the naive rolling σ approach at 3.0%. But GARCH is computationally heavier and more fragile (convergence issues). *Is the marginal accuracy worth the added complexity for a retail-scale portfolio?*

2. **Volatility Smile on Nifty Options:** OTM puts are consistently more expensive than BSM predicts. *Is the Nifty smile better explained by a jump-diffusion model (Merton, 1976) or a stochastic volatility model (Heston, 1993)?* I plan to fit both in Week 5 and compare residuals.

3. **Regime-Dependent Position Sizing:** My K-Means clustering identified 3 market regimes. *Should the Kelly fraction be computed separately per regime, or should it use a blended volatility estimate?* The theoretical answer (per-regime) may not be practical if regime transitions are detected with a lag.

4. **Adverse Selection in Limit Order Books:** Week 6 will cover market microstructure. *How much of the bid-ask spread on NSE is attributable to adverse selection vs. inventory risk?* Estimating this from publicly available LOB data (if accessible) would be a meaningful empirical exercise.

5. **Does Hierarchical Risk Parity (HRP) Outperform Mean-Variance in Regime Transitions?** HRP is touted as more robust to estimation error in the covariance matrix. *Is this advantage material for a 10–15 stock Nifty subset, or does it only matter for large institutional portfolios with hundreds of assets?*

6. **The Theta-Gamma Tradeoff as a Systematic Strategy:** My Week 4 simulation showed that selling Theta and hedging Delta can be profitable in low-volatility environments. *What is the long-run Sharpe ratio of this strategy on Nifty weeklies, and how badly does it blow up during tail events like the 2020 crash?*